

Inigo Quilez :: articles :: gradient noise derivatives - 2017



Intro

Similar to the [analytical derivatives of Value Noise](#), Gradient Noise accepts analytic computation of derivatives. And just like with Value Noise derivatives, this allows for much faster lighting computations or any other operation that requires gradients/normals based on the noise since, because no longer need to approximate it numerically (which often involve taking with multiple samples of the noise). It's also faster than using autodiff or dual numbers (thinks SLANG) because here we are reusing sub-expressions across both value and derivatives.

The code

Assuming we have some standard Gradient Noise implementation like the code on the left, the computation of the derivatives involves only a few more computations, as shown in the right.

```
// returns 3D gradient noise
float noise( in vec3 x )
{
    // grid
    vec3 i = floor(x);
    vec3 f = fract(x);

    // quintic interpolant
    vec3 u = f*f*f*(f*(f*6.0-15.0)+10.0);

    // gradients
    vec3 ga = hash( i+vec3(0.0,0.0,0.0) );
    vec3 gb = hash( i+vec3(1.0,0.0,0.0) );
    vec3 gc = hash( i+vec3(0.0,1.0,0.0) );
    vec3 gd = hash( i+vec3(1.0,1.0,0.0) );
    vec3 ge = hash( i+vec3(0.0,0.0,1.0) );
    vec3 gf = hash( i+vec3(1.0,0.0,1.0) );
    vec3 gg = hash( i+vec3(0.0,1.0,1.0) );
    vec3 gh = hash( i+vec3(1.0,1.0,1.0) );

    // projections
    float va = dot( ga, f-vec3(0.0,0.0,0.0) );
    float vb = dot( gb, f-vec3(1.0,0.0,0.0) );
    float vc = dot( gc, f-vec3(0.0,1.0,0.0) );
    float vd = dot( gd, f-vec3(1.0,1.0,0.0) );
    float ve = dot( ge, f-vec3(0.0,0.0,1.0) );
    float vf = dot( gf, f-vec3(1.0,0.0,1.0) );
    float vg = dot( gg, f-vec3(0.0,1.0,1.0) );
    float vh = dot( gh, f-vec3(1.0,1.0,1.0) );

    // interpolation
    return va +
        u.x*(vb-va) +
        u.y*(vc-va) +
        u.z*(ve-va) +
        u.x*u.y*(va-vb-vc+vd) +
        u.y*u.z*(va-vc-ve+vg) +
        u.z*u.x*(va-vb-ve+vf) +
        u.x*u.y*u.z*(-va+vb+vc-vd+ve-vf-vg+vh);
}
```

```
// returns 3D gradient noise (in .x) and its derivatives (in .yz)
vec4 noised( in vec3 x )
{
    // grid
    vec3 i = floor(x);
    vec3 f = fract(x);

    // quintic interpolant
    vec3 u = f*f*f*(f*(f*6.0-15.0)+10.0);
    vec3 du = 30.0*f*f*(f*(f-2.0)+1.0);

    // gradients
    vec3 ga = hash( i+vec3(0.0,0.0,0.0) );
    vec3 gb = hash( i+vec3(1.0,0.0,0.0) );
    vec3 gc = hash( i+vec3(0.0,1.0,0.0) );
    vec3 gd = hash( i+vec3(1.0,1.0,0.0) );
    vec3 ge = hash( i+vec3(0.0,0.0,1.0) );
    vec3 gf = hash( i+vec3(1.0,0.0,1.0) );
    vec3 gg = hash( i+vec3(0.0,1.0,1.0) );
    vec3 gh = hash( i+vec3(1.0,1.0,1.0) );

    // projections
    float va = dot( ga, f-vec3(0.0,0.0,0.0) );
    float vb = dot( gb, f-vec3(1.0,0.0,0.0) );
    float vc = dot( gc, f-vec3(0.0,1.0,0.0) );
    float vd = dot( gd, f-vec3(1.0,1.0,0.0) );
    float ve = dot( ge, f-vec3(0.0,0.0,1.0) );
    float vf = dot( gf, f-vec3(1.0,0.0,1.0) );
    float vg = dot( gg, f-vec3(0.0,1.0,1.0) );
    float vh = dot( gh, f-vec3(1.0,1.0,1.0) );

    // interpolation
    float v = va +
        u.x*(vb-va) +
        u.y*(vc-va) +
        u.z*(ve-va) +
        u.x*u.y*(va-vb-vc+vd) +
        u.y*u.z*(va-vc-ve+vg) +
        u.z*u.x*(va-vb-ve+vf) +
        u.x*u.y*u.z*(-va+vb+vc-vd+ve-vf-vg+vh);

    vec3 d = ga +
        u.x*(gb-ga) +
        u.y*(gc-ga) +
```

```

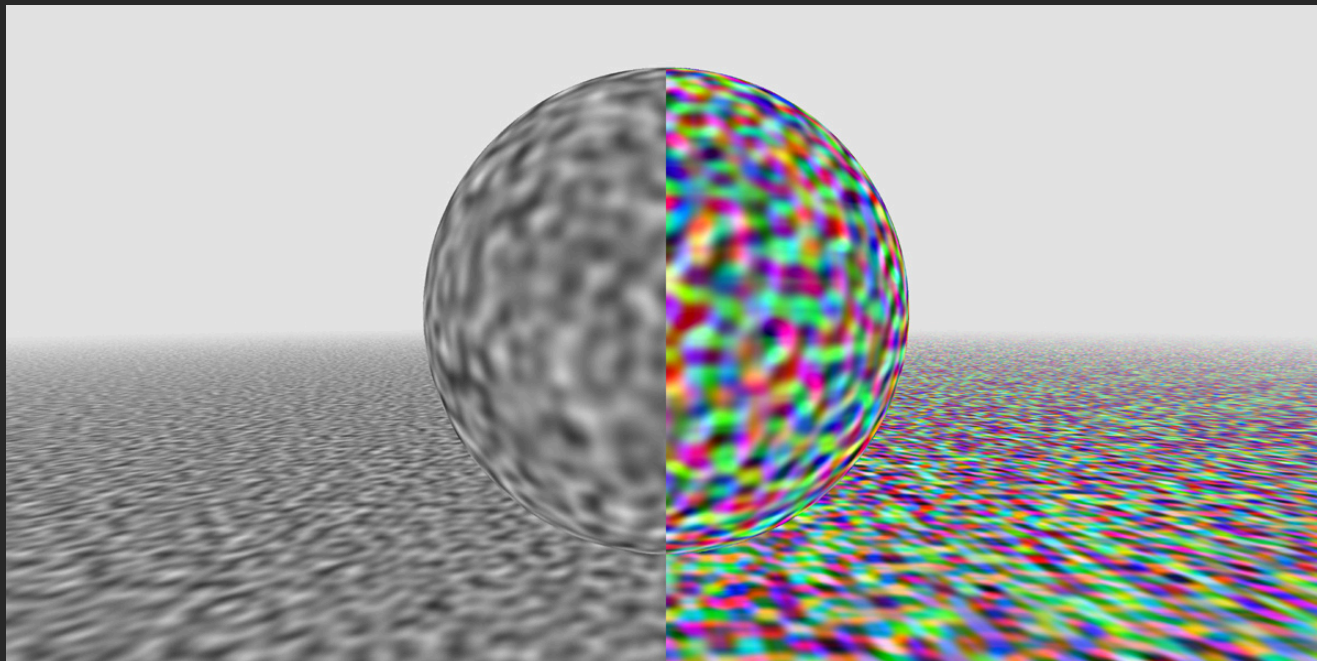
        u.z*(ge-ga) +
        u.x*u.y*(ga-gb-gc+gd) +
        u.y*u.z*(ga-gc-ge+gg) +
        u.z*u.x*(ga-gb-ge+gf) +
        u.x*u.y*u.z*(-ga+gb+gc-gd+ge-gf-gg+gh) +

        du * (vec3(vb-vb,vc-va,ve-va) +
               u.yzx*vec3(va-vb-vc+vd,va-vc-ve+vg,va-vb-ve+vd) +
               u.zxy*vec3(va-vb-ve+vf,va-vb-vc+vd,va-vc-ve+vg) +
               u.yzx*u.zxy*(-va+vb+vc-vd+ve-vf-vg+vh) );

    return vec4( v, d );
}

```

You can find a reference implementation here: <https://www.shadertoy.com/view/4dffRH>



Left: gradient noise, Right: derivatives of gradient noise

In the case of 2D, the code gets naturally smaller:

```

// returns 3D gradient noise (in .x) and its derivatives (in .yz)
vec3 noised( in vec2 x )
{
    vec2 i = floor( x );
    vec2 f = fract( x );

    vec2 u = f*f*f*(f*(f*6.0-15.0)+10.0);
    vec2 du = 30.0*f*f*(f*(f-2.0)+1.0);

    vec2 ga = hash( i + vec2(0.0,0.0) );
    vec2 gb = hash( i + vec2(1.0,0.0) );
    vec2 gc = hash( i + vec2(0.0,1.0) );
    vec2 gd = hash( i + vec2(1.0,1.0) );

    float va = dot( ga, f - vec2(0.0,0.0) );
    float vb = dot( gb, f - vec2(1.0,0.0) );
    float vc = dot( gc, f - vec2(0.0,1.0) );
    float vd = dot( gd, f - vec2(1.0,1.0) );

    return vec3( va + u.x*(vb-vb) + u.y*(vc-va) + u.x*u.y*(va-vb-vc+vd), // value
                ga + u.x*(gb-ga) + u.y*(gc-ga) + u.x*u.y*(ga-gb-gc+gd) + // derivatives
                du * (u.yx*(va-vb-vc+vd) + vec2(vb,vc) - va));
}

```

An implementation can be found here: <https://www.shadertoy.com/view/XdXBRH>

[inigo quilez](#) - learning computer graphics since 1994